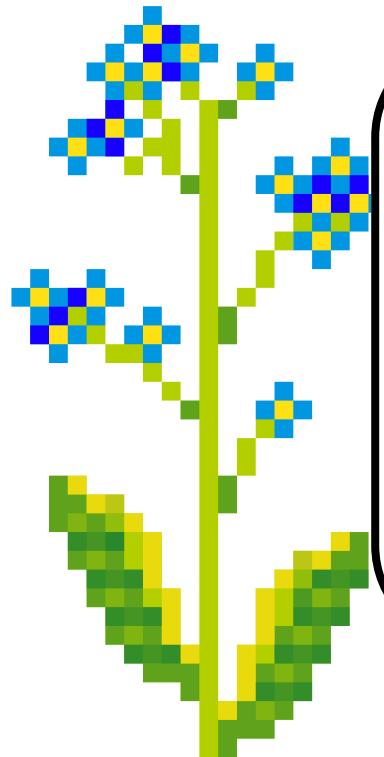




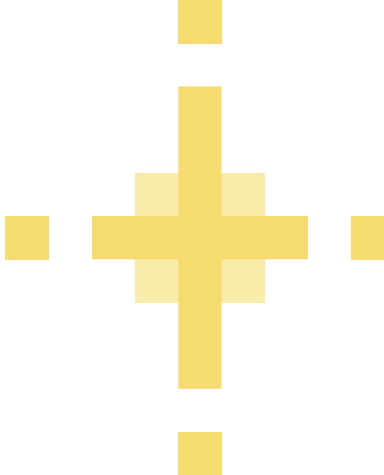
Eternal Clash 2

ปัญหาของโครงการ, กลุ่มเป้าหมาย และ, ขอบเขต

ปัญหา : เกมวางแผนบนเว็บส่วนใหญ่มักมีระบบที่ซับซ้อนเกินไป หรือใช้เวลาเล่นนานเกินไปจนไม่เหมาะกับเวลาว่างสั้นๆ

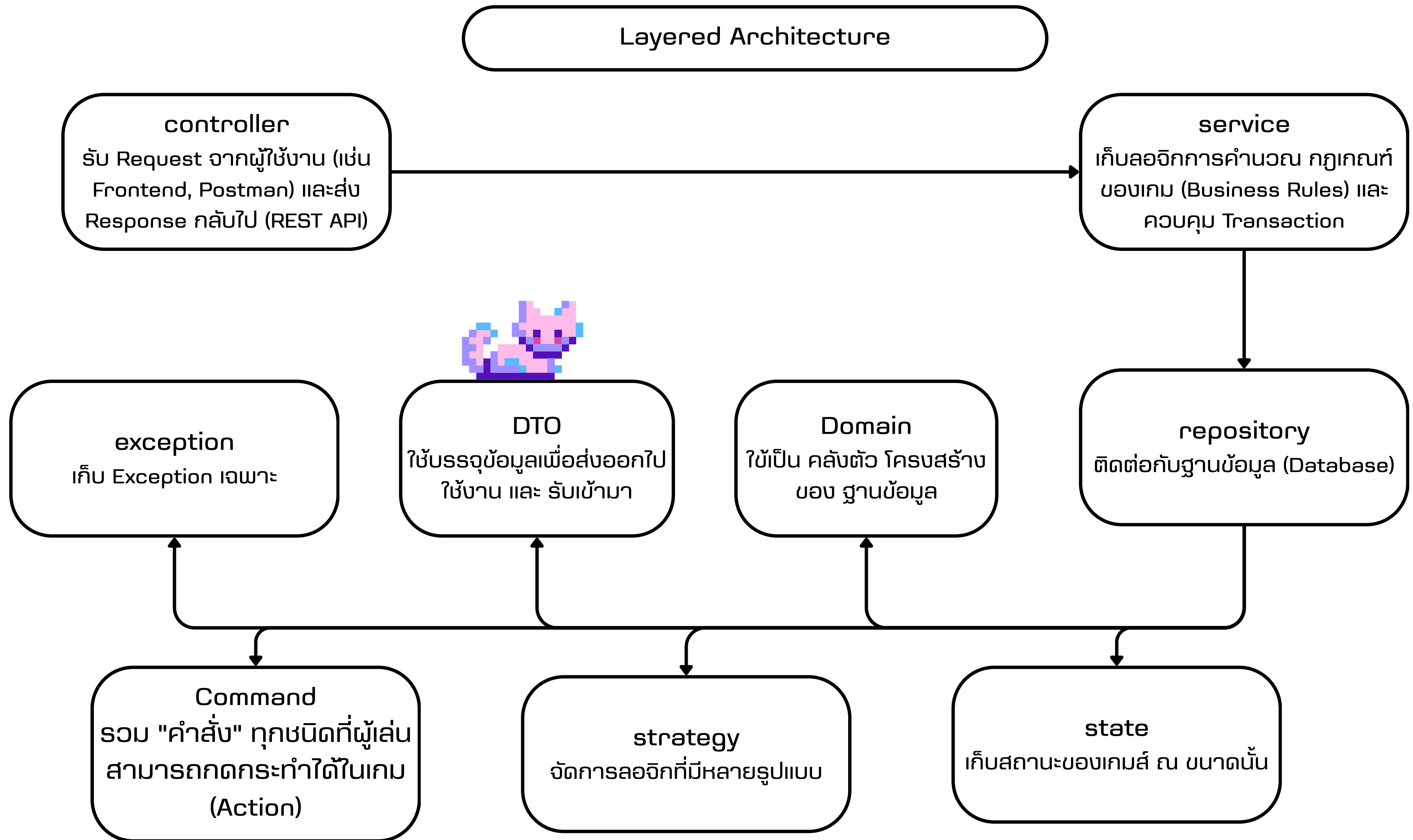


กลุ่มเป้าหมาย : ผู้เล่นที่ชื่นชอบเกมแนววางแผน (Strategy) แบบ Turn-based ที่ต้องการเกมจบไว เข้าใจง่าย และเล่นกับเพื่อนได้ทันที

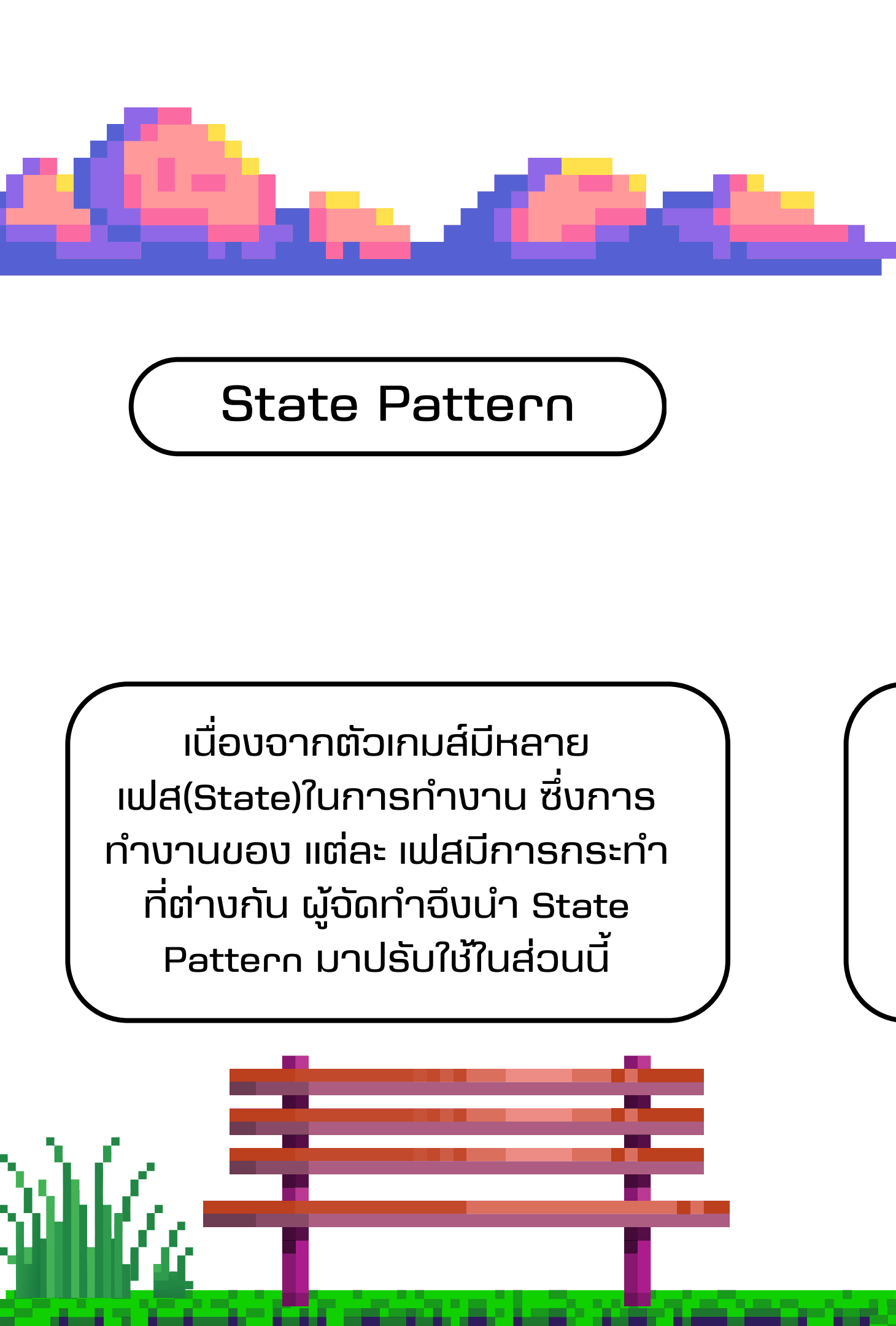


ขอบเขต

- Tech Stack: React (Vite) + Java Spring Boot + PostgreSQL (Supabase)
- รองรับผู้เล่นหลายคนแบบ Real-time ผ่านการแชร์ห้อง (Room Code)
- ระบบบุณพล (Marshal) ที่มีสกิลและกลยุทธ์ต่างกัน 7 ตัวละคร
- ระบบ Event ภัยพิบัติแบบสุ่ม และระบบต่อสู้คำนวณแพะชนะอัตโนมัติ



- S** GameClock.java รับผิดชอบคำนวณฤดู กลางวัน/กลางคืน และจุดสิ้นสุดฤดูโดยเฉพาะ ไม่ได้จัดการฐานข้อมูลหรือประมวลผลเทิร์น
- O** MarshalAbilityContext.java เลือกความสามารถผ่าน MarshalSpecialAbilityStrategy ที่อยู่ใน map จึงเพิ่มความสามารถใหม่ได้โดยไม่ต้องเพิ่มเงื่อนไข if-else ในบริการเกม อย่างไรก็ตาม ยังต้องลงทะเบียน strategy ใหม่ใน map จึงเป็นตัวอย่างของการรองรับการขยายได้บางส่วน
- L** KongmingCombatStrategy.java สืบทอดจาก StandardCombatStrategy และเพิ่มความสามารถเรื่องโอกาสรอดโดยยังใช้แทนคลาสพื้นฐานผ่าน CombatStrategy ได้ตราบดีที่ผู้เรียกใช้ยอมรับพฤติกรรมการรอดแบบสุ่มนี้
- I** PlayerActionCommand.java ทำให้คำสั่งเกมใช้ interface ร่วมกันได้ แต่มีจุดที่ควรปรับ: คำสั่งผลิตอาหารและเกณฑ์ทหารต้องมี getArmy() ทั้งที่ไม่มีกองทัพและจึงคืนค่า null
ProduceFoodCommand.java อาจแยก getArmy() ไปไว้ใน interface ย่อยสำหรับคำสั่งส่งทัพ
- D** รับ GameRepository, PlayerRepository และ repository อื่นผ่าน constructor ที่ Lombok สร้างให้จาก @RequiredArgsConstructor จึงพึ่งพา interface ของ repository แทนการสร้างคลาสเชื่อมฐานข้อมูลขึ้นมาเอง



Design Patterns

State Pattern

เนื่องจากตัวเกมส์มีหลาย
เฟส(State)ในการทำงาน ซึ่งการ
ทำงานของ แต่ละ เฟสมีการกระทำ
ที่ต่างกัน ผู้จัดทำจึงนำ State
Pattern มาปรับใช้ในส่วนนี้

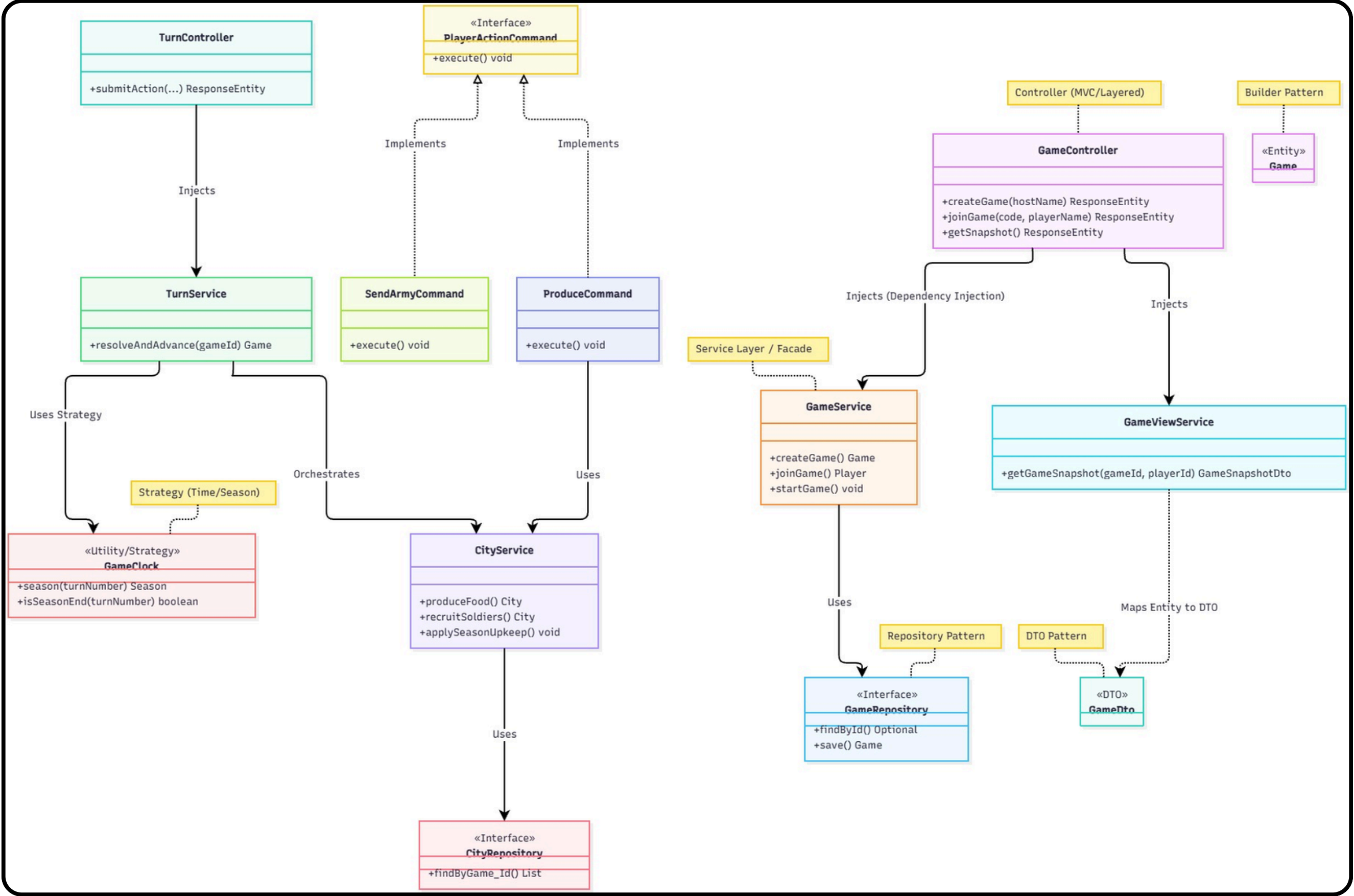
Command Pattern

Action แต่ละตัวต้องมีรูปแบบที่
ต่างกันและเมื่อในอนาคตจะมีการ
เพิ่ม อีกทั้ง Action จะทำงานจะ
ต้องรอ ให้รวบรวม Action ของ
ผู้เล่นทุกคนก่อน

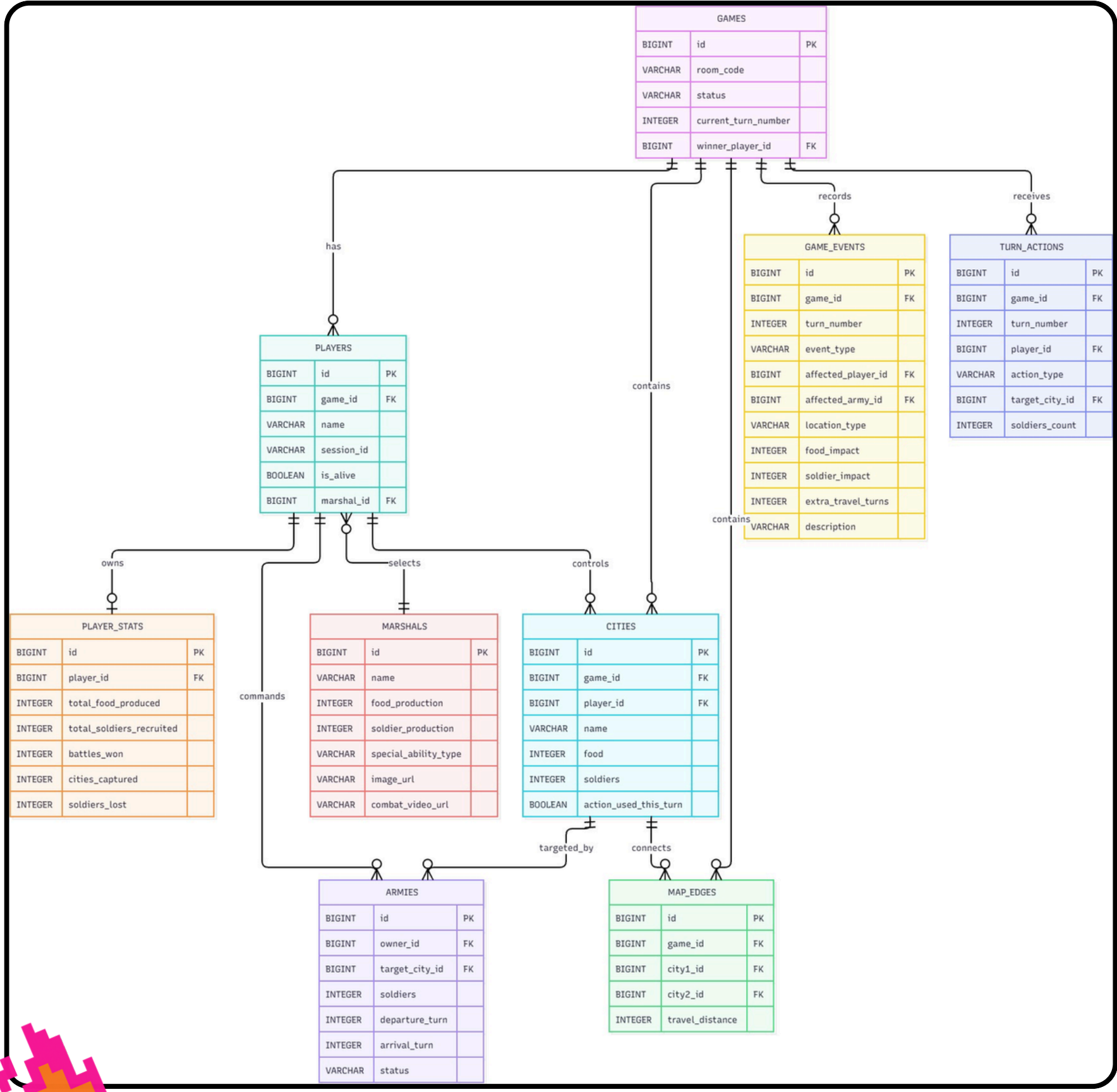
Strategy Pattern

เพราะ การคำนวณไม่ว่าจะทั้ง
การคำนวณความสูญเสีย หรือ
คำนวณ ผ่าน คลาส จอมพล
ทำให้การออกแบบ โดยเรียกใช้
Strategy Context จัดการในการ
เลือก Strategy ที่เหมาะสมกับ
งานนั้น

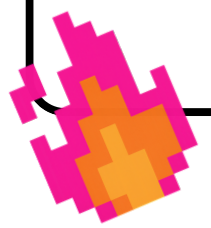
Class Diagram

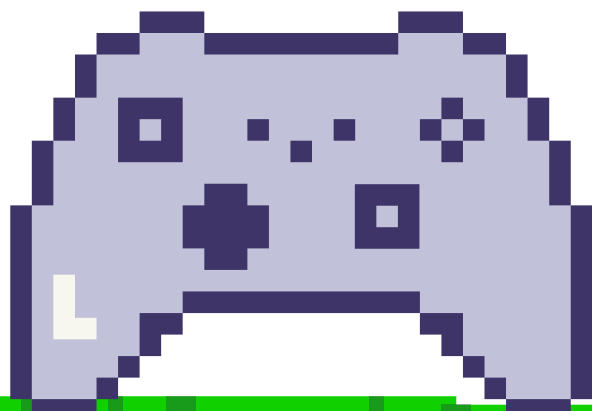
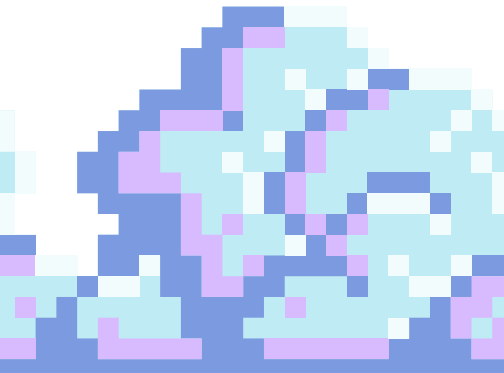
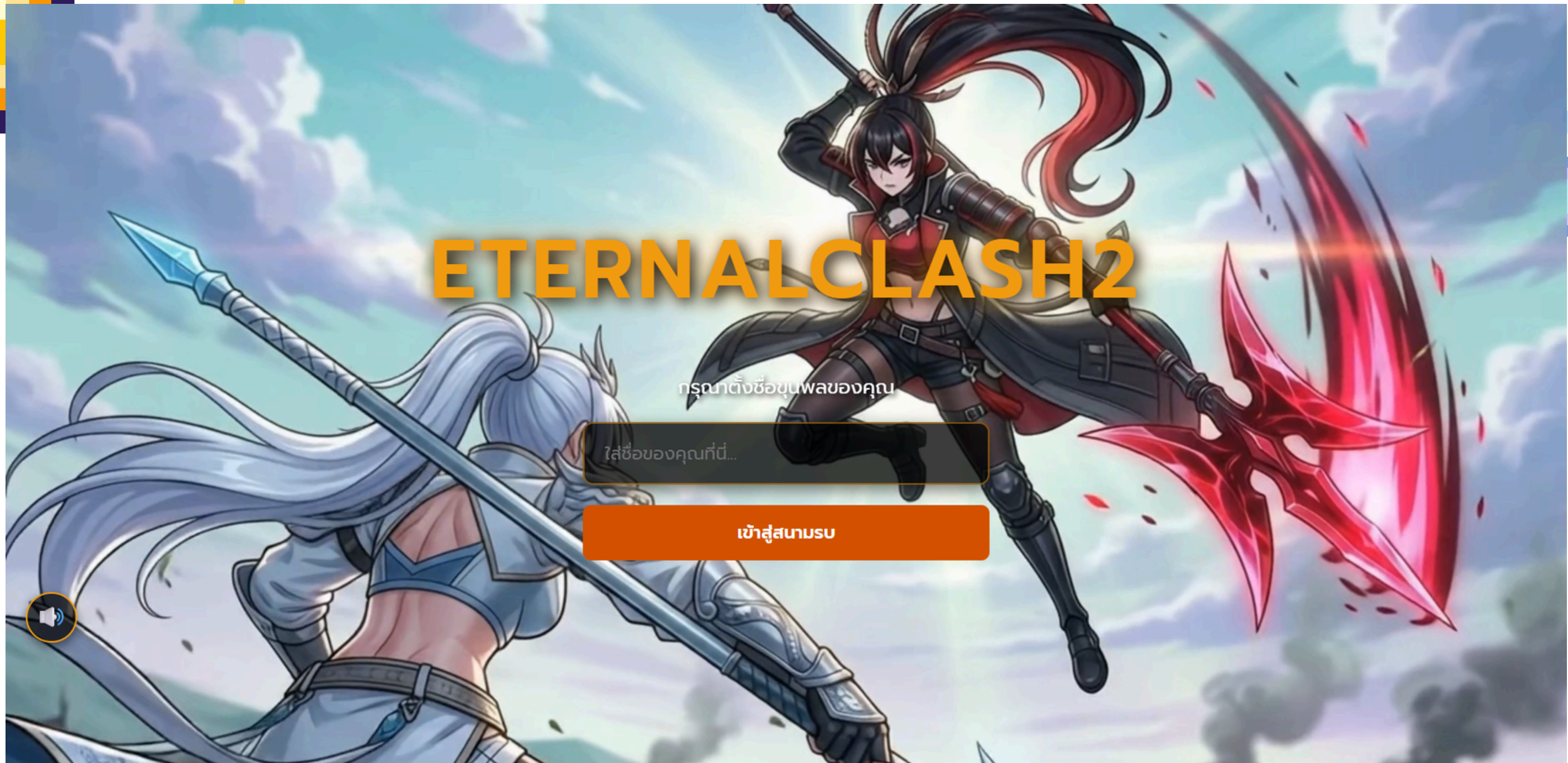
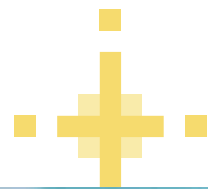


ER Diagram

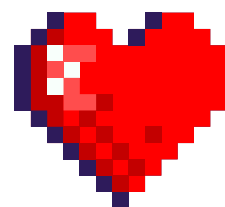
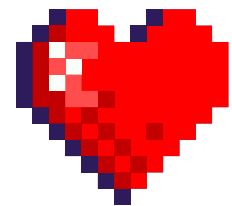


ความสัมพันธ์	ประเภท	คำอธิบาย
games → players	One-to-Many	1 เกมมีหลายผู้เล่น
players ↔ player_stats	One-to-One	1 ผู้เล่นมี 1 สถิติ (แยกเพื่อลดภาระ Query)
players → cities	One-to-Many	1 ผู้เล่นเป็นเจ้าของได้หลายเมือง (NULL = เมืองร้าง)
cities ↔ cities	Many-to-Many	เมืองเชื่อมต่อกันไปมาได้หลายทาง (ทำผ่านตารางกลาง map_edges)
players → armies	One-to-Many	1 ผู้เล่นส่งได้หลายกองทัพ
players → marshal candidates	One-to-Many	1 ผู้เล่นได้รับ 3 ตัวเลือกจอมพล
marshals → players	One-to-Many	1 จอมพลถูกเลือกโดยหลายผู้เล่น (คนละเกม)
marshals → marshal_candidates	One-to-Many	1 จอมพลเป็นตัวเลือกของหลายผู้เล่น
games → turn_actions	One-to-Many	1 เกมมีหลาย Action log
games → game_events	One-to-Many	1 เกมมีหลาย Event
games → battles	One-to-Many	1 เกมมีหลายการต่อสู้





Demo



ขอบคุณค่ะ

